面向 Python 加密代码的安全审计模型

第 1–6 轮训练、数据治理与评测实验报告

AI-Assisted Cryptographic Code Security Auditor

2026-10-05

摘要

本报告记录一个面向 Python 密码学误用的安全审计研究原型。系统以 13 条自有 Semgrep CRYPTO 规则作为静态候选生成器，以 QLoRA 微调的 Qwen3-4B 作为语义判别器，覆盖两个任务：**detect**（函数级判断是否漏洞、输出 CWE 与严重度）和 **triage**（给定代码与静态 finding，输出 Confirm/Reject、解释与可选修复）。第 1–4 轮完成了双任务基线、增量续训导致的灾难性遗忘、全量重训恢复以及合成负样本扩量实验；第 5 轮出现数据集 QA 失败和模型回归；第 6 轮把真实数据补强、规则夹具、强模型预审和浏览器人工复核合并进同一个全量重训轮次。最终 R6 从新鲜基座重训，产物包含 4,122 条训练/验证/测试记录，其中新增 401 条、真实记录 79 条、唯一真实 crypto 正例 30 条；冻结留出集 detect 为 19 条、triage 为 36 条（仅 4 条 Confirm）。R6 在 detect 上从 R5 的崩溃恢复，F1 达到冻结集 0.588、测试集 0.671，但未通过 precision、FPR 和 18 探针准确率等验收门槛，因此本报告将它定义为**可复现的科研基线**，而非生产级安全审计器。报告特别保留了第 6 轮的候选采集、预算估计、K3 二元审核、人工浏览器复核、来源标记、数据冻结与失败原因，供后续复习和 R7 设计使用。

目录

1 项目范围与报告目的 **3**

1.1 一句话介绍 3

1.2 两个任务 4

1.3 范围与非目标 4

2 基础知识：密码学、CWE 与代码审计 **4**

2.1 常见密码学构件 4

2.2 CWE 分类与 13 条 CRYPTO 规则 5

2.3 SAST、Semgrep、AST 与误报 5

2.4 detect 与 triage 的差别 6

2.5 LLM 微调基础 6

2.6 评估指标 7

3 系统架构、数据来源与来源标记 7

3.1 审计管线 7

3.2 数据层次 8

3.3 R6 最终数据快照 8

3.4 数据组成风险：类别平衡不等于有效信号 9

3.5 泄漏审计 9

4 第 1–5 轮：问题如何逐步暴露 10

4.1 第一轮：双任务基线 10

4.2 第二轮：triage 增量续训与灾难性遗忘 10

4.3 第三轮：fresh-base 全量合并重训 11

4.4 第四轮：负样本扩量与真实项目验证 11

4.5 第五轮：数据 QA 失败与模型回归 12

5 第六轮：数据优化、强模型审核与人工复核 12

5.1 R6 的修复目标 12

5.2 新增 322 条 curated/规则夹具数据 12

5.3 安全库对冲与参考层 13

5.4 真实候选池：从公告到函数级代码对 14

5.5 强模型预审：预算、模型选择与严格二元输出 15

5.6 浏览器人工复核平台 15

5.7 训练前冻结与复现约束 16

6 R6 训练与可复现信息 16

6.1 模型与平台 16

6.2 超参与配置差异 16

7 R6 评测结果 17

7.1 评测套件 17

7.2 冻结留出集 17

7.3 18 条真实 triage 探针 18

7.4 R6 设计测试集 18

7.5 13 规则能力矩阵 18

7.6 安全库抗误报 19

7.7 验收门槛 19

8 R6 相比 R5：恢复了什么，仍未解决什么 19

8.1 恢复 19

8.2 仍存在的问题 19

9 结论、局限与 R7 规划 19

9.1 结论 19

9.2 主要局限 20

9.3 R7 优化路线 20

10 复习问答与面试要点 21

10.1 常见问题 21

10.2 实验中的注意事项 22

A 复现命令与关键文件 22

A.1 构建与冻结 22

A.2 评测 22

A.3 关键产物 23

B 术语表 23

1 项目范围与报告目的

1.1 一句话介绍

本项目研究一个组合式代码安全审计原型：静态分析工具负责高召回地找出可疑密码学 API 使用；小型领域模型负责结合上下文判断 finding 是否真实、归类 CWE、解释原因并给出修复方向；不确定记录则进入强模型或浏览器人工复核。系统最终输出结构化 JSON 和 Markdown 审计报告。

项目的长期产品假设是“安全审计 copilot”：它不替代安全工程师，而是减少人工初筛时间，并把审计依据、数据来源和模型裁决保存为可追踪证据。

表 1: 系统的两个审计任务

任务	输入	输出
detect	一个 Python 函数或代码片段	`vulnerable`、`cwe`、`severity`、`confidence`、`explanation`
triage	代码 + 静态分析 finding	`cwe`、`severity`、`verdict=Confirm|Reject`、`explanation`、可选 `patch`

1.2 两个任务

1.3 范围与非目标

当前版本只覆盖已定义的密码学误用模式,包括弱哈希、破损算法、AES-ECB、可预测 IV/nonce、硬编码密钥、弱随机源、弱密钥长度、不安全 TLS、明文口令、弱 KDF 参数和非常量时间比较。它不声称能够发现未知密码学漏洞类别，也不生成攻击或利用代码。

本报告明确不作以下声明:

- 不声称模型达到生产级 precision、recall 或 CWE 路由能力;
- 不把强模型 K3 的审核结果改写为人工 ground truth;
- 不声称训练合并制品零泄漏;
- 不把 R6 的测试集边界值包装成真实 crypto 泛化能力;
- 不把“训练更久”视为当前瓶颈的解决方案。

2 基础知识：密码学、CWE 与代码审计

2.1 常见密码学构件

理解密码学代码审计，需要先把“算法本身”与“算法在业务中的用途”分开。一个弱原语并不自动等于漏洞：MD5 用于口令存储通常是缺陷，用于缓存键或变更检测通常不是安全缺陷。模型最难学的正是这种上下文边界。

IV（initialization vector）和 nonce（number used once）是许多模式安全性的前提。CBC 类模式要求 IV 不可预测；GCM 等流式 AEAD 要求 nonce 在同一密钥下绝不重复。仅看到“函数使用了 AES”不足以判断安全与否，必须同时检查模式、IV/nonce、密钥来源和调用用途。

口令哈希不能与普通哈希混为一谈。直接 `sha256(password)` 很快，适合校验文件但不适合抵抗离线爆破。密码系统应使用带盐、可调工作量的 KDF 或口令哈希；现代选择包括 Argon2id、scrypt、bcrypt 或按当前标准配置的 PBKDF2。审计时还要判断迭代次数是否过低、盐是否随机且唯一、参数是否可由攻击者控制。

表 2: 密码学构件与审计关注点

构件	作用	常见审计风险
对称加密	用同一密钥加密和解密，如 AES	ECB 模式、固定 IV、nonce 复用、弱密钥长度
AEAD	同时提供机密性和完整性，如 AES-GCM、ChaCha20-Poly1305	nonce 复用会破坏认证；密钥来源和 nonce 生命周期错误
非对称加密与签名	公钥加密、私钥签名，如 RSA、ECDSA、Ed25519	RSA 位数不足、填充错误、私钥或随机源问题
哈希	把任意长度输入映射为固定长度摘要，如 SHA-256	用 MD5/SHA-1 保护安全敏感内容；把普通摘要误当口令哈希
MAC/HMAC	用密钥提供消息完整性和真实性	非常量时间比较、密钥复用、算法降级
KDF 与口令哈希	从口令派生密钥或抵抗离线爆破，如 PBKDF2、scrypt、Argon2	迭代次数过低、盐缺失、参数由攻击者控制
随机数	生成密钥、token、salt、IV、nonce	使用 `random`（Mersenne Twister）而非 `secrets` 或 `os.urandom`
TLS 与证书	验证服务器身份并建立加密信道	`verify=False`、`CERT_NONE`、关闭 hostname check、信任根被覆盖

2.2 CWE 分类与 13 条 CRYPTO 规则

项目将静态规则和 CWE 目标绑定。CWE 是缺陷类型，规则是工程上可检测的模式；两者不能完全等同。一个规则可以覆盖多个 CWE，一个 CWE 也可能需要多条规则。

报告重点覆盖 9 个目标 CWE: CWE-321（硬编码密钥）、CWE-326（加密强度不足）、CWE-327（破损或危险算法）、CWE-329（未使用随机 IV/nonce）、CWE-338（弱 PRNG）、CWE-916（口令哈希计算量不足）、CWE-208（可观测时序差异）、CWE-295（证书验证不当）和 CWE-256（明文存储口令）。R6 候选池中还会遇到 CWE-798（硬编码凭据）、CWE-330（随机性不足）、CWE-328（可逆单向哈希）等相邻类别，但相邻类别不应未经规则映射就直接进入目标训练集。

2.3 SAST、Semgrep、AST 与误报

SAST（static application security testing）在不运行程序的情况下分析源代码。Semgrep 以模式和轻量语义匹配生成 finding；Python AST 可用于提取函数、比较结构、定位调用和检测近似重复。

静态分析的优势是快、可解释、可定位；缺点是模式命中不等于真实缺陷。例如：

- 密码库实现 MD5 或 DES 是为了兼容 legacy 协议或历史格式，这是“有效的误报”；
- 测试夹具、缓存键、ETag、一致性校验、分片和 jitter 使用弱原语，通常没有安全目标；
- 一个函数虽然调用了 AES，但使用了固定 IV、ECB 或可预测 nonce，仍然是真实问题。

因此系统采用“静态召回 + LLM 语义分流”：静态规则作为候选生成器，LLM 负责判断用途

表 3: 13 条 CRYPTO 规则族与核心 CWE

规则	问题	目标 CWE
CRYPTO-001	MD5 等弱摘要用于安全敏感场景	CWE-327 / CWE-916
CRYPTO-002	SHA-1 等弱摘要用于安全敏感场景	CWE-327 / CWE-916
CRYPTO-003	DES 或 3DES	CWE-327
CRYPTO-004	RC4	CWE-327
CRYPTO-005	AES-ECB	CWE-327
CRYPTO-006	可预测或复用 IV/nonce	CWE-329
CRYPTO-007	硬编码密钥或口令	CWE-321
CRYPTO-008	弱伪随机数生成器	CWE-338
CRYPTO-009	弱密钥长度或弱曲线	CWE-326
CRYPTO-010	不安全 TLS 或证书校验	CWE-295
CRYPTO-011	明文口令存储或记录	CWE-256
CRYPTO-012	弱 KDF 参数	CWE-916
CRYPTO-013	非常量时间比较	CWE-208

和安全边界。LLM 可以减少误报，但也可能幻觉出代码中不存在的缓存、测试或 ETag 语境，所以必须有结构化 schema、评估集和来源审计。

2.4 detect 与 triage 的差别

detect 是开放判断：模型只看到代码，要决定是否 vulnerable，并给出 CWE。triage 是受限判断：模型同时看到代码和一条具体 finding，要决定 Confirm 或 Reject。两者的监督信号不同：

- detect 需要安全代码、真实漏洞、规则命中与规则未命中的负例；
- triage 需要“规则命中但非安全用途”的 Reject，以及规则命中且真实安全敏感的 Confirm；
- detect 的训练分布会影响模型是否“宁可全报”；
- triage 的训练分布会影响模型是否把弱原语用途过度泛化为 Reject。

2.5 LLM 微调基础

本项目使用监督微调（SFT）和 QLoRA。SFT 把输入 prompt 与期望 JSON 输出作为训练对；LoRA 冻结基座权重，只训练低秩适配矩阵；QLoRA 进一步用 4-bit 量化降低显存需求。核心超参如下：

灾难性遗忘在本项目反复出现。R2 只喂 54 条 triage 增量，探针 FPR 降为 0，但 detect CWE accuracy 从 0.796 掉到 0.433，domain Confirm recall 从 0.357 掉到 0。R4 微量续训更严重，detect recall 从 0.990 降到 0.584，domain Confirm recall 从 0.929 降到 0.071。由此得到的工程约束是：**任务或分布明显变化时必须 fresh-base 全量合并重训；增量续训只适合同分布的小修**。

表 4: 微调术语

术语	含义与影响
rank（r）	LoRA 低秩矩阵的秩；越大容量越高，也越容易过拟合或干扰旧能力
alpha	LoRA 缩放系数；通常与 rank 成比例，R1–R6 主要使用 rank=16、alpha=32
dropout	适配层随机丢弃率；用于抑制过拟合
learning rate	更新步长；过大破坏基座能力，过小则学不到新边界
epochs	数据重复训练次数；不代表越多越好，可能强化模板和噪声
max length	截断长度；过长样本会被截断，导致代码上下文缺失
fresh-base retrain	从原始基座用全量合并数据重训；本项目最稳定的范式
incremental update	在原模型上只喂新增样本续训；R2/R4 两次引发灾难性遗忘
catastrophic forgetting	新任务学会后，旧任务能力显著退化

2.6 评估指标

设 TP、FP、FN、TN 分别表示真阳性、假阳性、假阴性、真阴性。detect 的主要指标为:

$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}, \tag{1}$$

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \qquad \text{accuracy} = \frac{TP+TN}{TP+FP+FN+TN}. \tag{2}$$

CWE accuracy 只在 ground-truth 正例上计算，表示漏洞已被判定为 vulnerable 时，CWE 是否完全正确。triage 的 Confirm recall 表示真实 Confirm 中被保留的比例；FPR（false positive rate）表示真实 Reject 中被错误 Confirm 的比例:

$$\text{FPR} = \frac{FP}{FP+TN}. \tag{3}$$

小正例数会让 recall 极不稳定。冻结 triage 只有 4 条 Confirm，一条预测变化就会改变 25 个百分点；因此本报告只把冻结 Confirm recall 当作计数，不作为验收门槛。同样，30 条真实 crypto 正例低于项目预设的 50–100 条门槛，所以不能从聚合分数声称真实 crypto 泛化能力。

3 系统架构、数据来源与来源标记

3.1 审计管线

Python 源码 → 13 条 Semgrep CRYPTO 规则 → 候选 finding → Qwen3-4B 微调模型（detect / triage）→ 结构化 JSON → Markdown 审计报告

主要模块包括 `rules/`、`analyzer/`、`agent/`、`model/`、`evaluation/`、`scripts/` 和 `reviewer/`。训练与推理由同一套 prompt 模板控制，以保证 chatml 训练数据与部署输入分布一致。

3.2 数据层次

表 5: 训练数据的语义层次

层次	典型来源	用途与限制
公共漏洞数据	PyCode-Vul	提供 Python 通用漏洞 detect 信号；多数是 Web/应用漏洞，不是 crypto 专用真值
领域规则夹具	Semgrep、Bandit 测试桩	提供规则命中、安全替代写法、Reject/负例；`verified=false`，不是真实漏洞样本
人工/模型构造样本	curated、组合模板	补齐 13 个规则族和类别比例；可能模板化，需防止过拟合
真实 advisory 记录	GHSA、OSV/PyPI、PyPA、fix commit	有漏洞、修复函数对和来源链；是当前最有价值的真实 crypto 信号
安全参考	固定 commit 安全库代码	用于修复建议和安全写法比较，不直接进入训练，除非满足复核门禁

3.3 R6 最终数据快照

R6 将真实记录直接合并进第六轮，不另建第七轮。最终上传集为：

表 6: R6 最终数据划分

Split	detect	triage	合计
train	2732	383	3115
val	299	48	347
test	602	58	660
合计	**3633**	**489**	**4122**

R6 相比 R5 基座新增 401 条：

79 条真实记录中，60 条来自 K3 模型审核，19 条来自浏览器人工审核。模型审核记录保留 `review_mode=model` 和 `model=kimi-k3`，人工记录保留 `review_mode=manual`。即使项目允许 K3 标签作为最终标签，报告也不能把它写成人工审核。

真实 detect 正例的 CWE 分布为：CWE-295（12）、CWE-208（9）、CWE-327（3）、CWE-338（2）、CWE-321（2）、CWE-326（1）、CWE-256（1）。真实 triage 导出为 30 Confirm / 19 Reject，主要信号集中在 CRYPTO-010（12 Confirm）和 CRYPTO-013（9 Confirm）。

表 7: R6 新增记录来源

来源层	条数	verified 与用途
curated、规则夹具与补充样本	322	全部 `verified=false`；用于规则覆盖、负例/Reject 和工具回归
真实 advisory detect 记录	30	全部漏洞正例；23 条 K3、7 条人工
真实 advisory triage 记录	49	30 Confirm / 19 Reject；37 条 K3、12 条人工
合计	**401**	真实记录 79 条；唯一真实 crypto 正例仍只有 30 条

3.4 数据组成风险：类别平衡不等于有效信号

R6 detect 有 2,037 条正例、1,596 条负例；表面比例合理，但只有 434 条正例属于目标 crypto CWE，其余 1,603 条是通用 Web/应用/内存漏洞。也就是说，模型大部分正监督花在非 crypto 缺陷上。目标 CWE 的样本量为：

表 8: 目标 crypto CWE 在 detect 正例中的样本量

CWE	正例数	CWE	正例数
CWE-327	275	CWE-916	11
CWE-326	63	CWE-256	10
CWE-295	33	CWE-321	5
CWE-208	18	CWE-329	5
CWE-338	14		

稀有 CWE 的逐类召回会有很高方差。训练时如果按总样本随机采样，稀有类和真实 advisory 正例会进一步被稀释；未来需要 train-only 的 crypto-focus sampler，并按 CWE 与规则设置最低配额。

3.5 泄漏审计

冻结评测集的目标是移除所有出现在 R5/R6 最终 train 或 val 中的空白归一化代码和 AST 完全相同样本。冻结后的 R6 评测集为：

- detect：19 条（7 正例 / 12 负例），删除 0 条；
- triage：36 条（4 Confirm / 32 Reject），删除 6 条与 `round4-real-passlib` 重复的记录。

但是，合并制品的跨 split 审计仍发现继承自 R4/R5 的残余重复：

因此只能声称“冻结评测集不暴露给优化可见集”，不能声称整个合并训练集零泄漏。后续新增数据必须继续按仓库和 commit 切分，不能随机按行划分。

表 9: 合并训练制品的残余近似重复

配对	完全重复行	AST 相同行
train / val	2	21
train / test	2	17
val / test	2	2

4 第 1–5 轮：问题如何逐步暴露

4.1 第一轮：双任务基线

目标：在公共 Python 漏洞数据与领域密码数据上做首次领域适配，跑通 detect/triage 全链路。

数据：detect 2,406条 + triage 132 条 = 2,538 条。

关键配置：LoRA rank=8，学习率 1e-4。

结果：

- detect CWE accuracy 0.796，F1 0.656；
- domain Confirm recall 0.357；
- 18 条 triage 探针 Confirm recall 0.917，FPR 0.667，accuracy 0.722。

暴露的问题：模型在通用 detect 上可用，但 triage 会把非安全用途的弱原语大量判为 Confirm。典型误报包括缓存键中的 MD5、变更检测中的 SHA-1、退避抖动中的 `random`、测试夹具和 legacy 兼容实现。FPR 0.667 说明“规则命中”被错误当成“真实安全缺陷”。

4.2 第二轮：triage 增量续训与灾难性遗忘

目标：只补 triage 增量样本，压降 FPR。

数据：54 条（28 Confirm + 26 Reject），只含 triage。

关键配置：batch=8，学习率 1e-4，3 epochs，rank=16 / alpha=32 / dropout=0.05，max length=1024。

结果：

- 探针 FPR 从 0.667 降到 **0.000**，accuracy 从 0.722 升到 0.944；
- detect CWE accuracy 从 0.796 降到 **0.433**，343/569 条样本 CWE 预测偏移；
- domain Confirm recall 从 0.357 降到 **0.000**，模型几乎全判 Reject。

教训：只喂新任务的增量数据会破坏旧任务表征。短期目标指标改善，不等于模型整体能力提升。增量续训从此被列为高风险路线。

4.3 第三轮：fresh-base 全量合并重训

目标：放弃 R2 续训，从基座用全量合并数据重训，同时恢复 detect 与修复 triage。

数据：2,595条 = R1 的 2,538 条 + R2 的 54 条 + 3 条签名/认证修补。

关键配置：batch=16，学习率 1e-4，3 epochs，rank=16 / alpha=32 / dropout=0.005，cosine，max length=2048。

关键决策：在 triage prompt 中追加 `TRIAGE_PURPOSE_GUIDE`，要求模型明确判断 flagged call 的用途是安全目的还是非安全目的。无引导时同一模型 FPR 为 0.500；有引导时 FPR 为 0.000，Confirm recall 保持 1.000。

结果：

- detect recall 0.990，precision 0.537，F1 0.697，CWE accuracy 0.921；
- domain Confirm recall 0.929，FPR 0.067，accuracy 0.931；
- 18 探针 accuracy 18/18。

遗留问题：detect 仍近乎全报，562/569 判为漏洞，`tp/fp/fn/tn = 302/260/3/4`。precision 0.537 几乎等于正例比例 0.536，说明二元 vulnerable 判定缺少判别力。根因是 detect 负例几乎都是非加密 Django 代码，模型从未见过“安全的加密代码”。

4.4 第四轮：负样本扩量与真实项目验证

目标：补安全加密负例，压降 detect 过报，并用真实密码库做留出验证。

负样本设计：

- A 类：现代安全写法，如 scrypt、Argon2、bcrypt、PBKDF2、AES-GCM + 随机 nonce、ChaCha20-Poly1305、SHA-3、HKDF、ECDH、`secrets`/`os.urandom`、RSA-OAEP、Ed25519；
- B 类：弱原语用于非安全目的，如缓存键、去重、分片、变更检测、退避抖动、测试夹具；
- strong-B：规则会命中，但模型必须结合用途判为非漏洞；
- weak-B：规则不命中，用于普通安全负例。

两条路线：

结果：

- 全量 50:50：detect recall 0.984，precision 0.536，F1 0.694，CWE accuracy 0.897；fp=260、tn=4 与 R3 基本不变；
- 微量续训：detect recall 0.990 → 0.584，domain Confirm recall 0.929 → 0.071，灾难性遗忘再次出现；

表 10: R4 增量续训与全量重训

	微量续训	**全量 fresh-base**
起点	R3 模型	原始基座
数据	77 条负样本	3,315 条（detect 3126 + triage 189）
负样本	42 A + 35 B	新增 150 正 + 493 负，最终 1563:1563
lr / epochs	5e-5 / 2	1e-4 / 3

- 42 条真实项目验证：detect 的 7 条安全函数只有 4/7 判对；domain triage FPR 为 0.429（微量）/0.571（全量）；passlib legacy-hash 11 条全部被误判 Confirm。

教训：合成负样本数量并没有教模型真实库的安全边界。模板化负例容易被记忆，不能替代真实库代码与协议兼容场景。

4.5 第五轮：数据 QA 失败与模型回归

R5 的原意是继续修复数据分布和 triage 能力，但这一轮产生了明确回归。已提交的 R5 运行在 detect 切片上为 `tp/fp/fn/tn = 0/2/3/8`，F1 为 0.000，accuracy 0.615；triage n=41、3 Confirm / 38 Reject，FPR 0.368。R5 数据集 QA 报告为 FAIL，包含：

- `backtest_pos_missing=22`：应有 positive 回测信号的样本缺失；
- `backtest_bneg_no_fire=25`：strong-B 负例未触发预期规则；
- 同一配置重跑得到 detect `0/1/3/9`、triage FPR 0.289，说明点估计存在运行波动。

教训：数据质量门禁必须优先于训练。R5的回归既有数据问题，也有运行波动；不能把单次指标当成稳定能力，也不能让坏数据进入下一轮而不修复。

5 第六轮：数据优化、强模型审核与人工复核

5.1 R6 的修复目标

R6 针对 R5 的四个问题设计：

5.2 新增 322 条 curated/规则夹具数据

322 条基线新增来自 8 个固定源文件，构成如下：

其中 detect 为 216 条（130 正 / 86 负），triage 为 185 条（113 Confirm / 72 Reject）是连同真实记录后的口径；基线 322 条中，detect 186 条（100 pos / 86 neg），triage 136 条（83 Confirm / 53 Reject）。

规则夹具主要来自：

表 11: R5 问题与 R6 对策

R5 问题	R6 对策	执行结果
triage Confirm 召回下滑	增加 Confirm 样本，保持二元复核	R6 新增 triage Confirm 113 / Reject 72
domain FPR 反弹	扩增 Reject、安全库对冲	冻结 FPR 0.125，但仍高于 0.06 目标
triage 类别失衡	按规则族和类型补样	13 个规则族均覆盖
detect 规则覆盖不足	加入规则夹具和真实 advisory 记录	13/13 规则族覆盖；真实正例 30 条仍偏少

表 12: R6 基线新增 322 条来源

源文件	条数
`new_detect.jsonl`	33
`new_triage.jsonl`	21
`supplement_detect.jsonl`	28
`supplement_triage.jsonl`	5
`supplement_v2_detect.jsonl`	21
`supplement_v2_triage.jsonl`	6
`rule_fixture_detect.jsonl`	104
`rule_fixture_triage.jsonl`	104
合计	**322**

- Semgrep 官方规则测试，使用 `\# ruleid:` / `\# ok:` 标记；许可为 Semgrep Rules License v1.0；
- Bandit 示例程序，许可为 Apache-2.0；
- 只映射与 CRYPTO-001 至 CRYPTO-013 语义一致的规则，跳过 blowfish、RC2、IDEA、XOR、MD2、MD4、JWT、sha224 等更宽泛模式；
- 正例必须能命中对应规则 scope，负例必须明确使用安全替代写法，如 SHA-256、AES-GCM、`os.urandom`、`create_default_context`。

这 208 条规则夹具全部是 `verified=false`。它们是工具回归和 finding 覆盖数据，不是真实漏洞正例；如果对外再分发，需要遵守 Semgrep 来源行的许可。

5.3 安全库对冲与参考层

R4/R5 的一个硬问题是安全库代码大量被误判。R6 增加了多类安全库负例，包括 cryptography、PyNaCl、itsdangerous、PyJWT、PyCryptodome、passlib、argon2-cffi、ecdsa、google-auth、oscrypto。其目标不是“因为项目安全所以函数安全”，而是逐函数检查参数、用途、密钥来源和上下文。

`result.txt` 中的 15 条固定 commit 审核候选，有 7 条被标为 `secure_reference`，写入独立参考层；它们保持 `verified=false` 和 `training_eligible=false`，只用于修复建议、安全写法比较和后续人工晋级，不进入训练。另外 8 条建议 exclude，不合并。

5.4 真实候选池：从公告到函数级代码对

R6的数据工程不是“下载一个数据集”，而是构建候选、筛选、补代码、审核、合并的可追溯流水线：

1. 从 GHSA、OSV/PyPI、PyPA、PyCode-Vul 和本地 JSON/JSONL 采集候选；
2. 按 CWE、规则、包名、commit、repo 和关键词筛选，去重后形成候选池；
3. 用 fix commit 抓取 vulnerable/fixed 代码对；能 `ast.parse` 的 366 条更适合函数级训练；
4. 按证据强度分 P1/P2/P3；P1/P2 进入优先复核队列；
5. 用强模型做严格二元预审，或进入浏览器人工复核；
6. 只有 Confirm/Reject 结论进入 `real_verified_detect.jsonl` 和 `real_verified_triage.jsonl`；
7. 用 builder 合并 R5 基座，生成 train/val/test 和 SHA-256 manifest。

离线执行后的关键快照：

表 13: R6 候选池快照

指标	值
已落盘外部 advisory 源	GHSA 约 38 万 JSON、OSV-PyPI 约 2.6 万 JSON、PyPA 7,721 YAML、Bandit、Semgrep、Trail of Bits 规则
各源 harvester 输出	GHSA 534、OSV-PyPI 843、PyPA 706、PyCode-Vul 389
合并与去重	2,718 条合并，去重后 1,861 条入选
P1 / P2 / P3	5 / 2 / 1,854
已带代码对	400 条，其中 366 条两端可 `ast.parse`
仅缺代码	549 条有 fix commit
缺 commit 与 repo	703 条，需先补包到仓库映射
B 类训练量	331 条可解析真实生产代码对，无 advisory
完整 P3 强模型估价	K3 high 约 975.18 元，K2.7-code high 约 267.01 元；不可全量盲跑

这份快照本身是重要事实：R6 的瓶颈不是“没有公告”，而是公告到可训练函数对之间的代码、commit、仓库和人工证据缺口。

5.5 强模型预审：预算、模型选择与严格二元输出

卷积式全量审核不现实，因此项目先估价再调用。A 类函数候选批次的 K3 审核实际使用情况为：

表 14: K3 真实候选审核成本

项目	值
候选数	60
prompt tokens	240,546
completion tokens	26,995
实际输入成本	4.81092 元
实际输出成本	2.69950 元
实际总成本	**7.51042 元**
模型 / reasoning effort	`kimi-k3` / `high`
价格口径	输入 20 元/百万 token，输出 100 元/百万 token

估价阶段曾用保守口径估算 high reasoning 约 33.59 元；实际输出远低于假设上限。这个差异说明“先估价、后调用、再按实际 usage 回报”是必要流程，不能凭候选数量猜测费用。

审核提示词采用严格二元格式：

- 只允许 `confirmed` 或 `rejected`；
- 证据不足时强制 `rejected`，不引入 uncertain；
- API、解析或 schema 错误记录为 error，不导入；
- 导入后保留 `review_mode=model`、`model=kimi-k3`、`model_confidence` 和 `source_audit`；
- 不允许把 60 条 K3 结果写成人工 ground truth。

导入报告显示：审核文件 60 行，增量导入时已有 48 条已复核、1 条新导入；8 条缺少 vulnerable code、3 条 vulnerable snippet 不是有效 Python，均被跳过，没有进入已验证导出。最终 K3 来源为 23 条 detect + 37 条 triage。

5.6 浏览器人工复核平台

为减少人工工作量，系统只把不确定和抽样部分放进浏览器审核。最终 UI 刻意简化为两个按钮：

- **Confirm**：导出一条 detect positive 和一条 triage Confirm；
- **Reject**：导出一条 triage Reject；
- 不再提供 uncertain、scope-out 或 license 字段；

- Confirm 要求目标规则、目标 CWE、解释和两段能 `ast.parse` 的 Python 片段；
- Reject 要求 vulnerable snippet 和解释；
- 审核员可在浏览器中修正 snippet，但不能凭空录入未经来源支持的样本。

每条 verdict 是源真相，存放于：

```
data/round6/review/web/<reviewer>/labels/<verdict>/<candidate_id>.json
```

应用据此重建 `real_verified_detect.jsonl` 和 `real_verified_triage.jsonl`。最终 19 条人工复核记录为 7 条 detect + 12 条 triage；其中一条候选因代码长度超出允许范围被排除。人工标签与模型标签共用相同导出 schema，但来源字段不同。

5.7 训练前冻结与复现约束

R6 在训练前生成 `r6_manifest.json`，冻结来源文件和 train/val/test 的 SHA-256。关键制品为：

表 15: R6 上传制品哈希

制品	行数	SHA-256
train	3115	139966e5217d4b2976db7a13f86d70ecb8aad09adc1198092f782e028041bd4d
val	347	6b6797b5fb1da66c797f2efe28dc7559d124b2cd4adb5f75496a40f07d397f4b
test	660	1549c1d7d5c8484aafe677ddf83ccfbdf6912e6052d1e910d49f45ed66c7d64c

冻结后不允许修改 train/val/test。评测使用 `validation_frozen/`，训练期间不可见。

6 R6 训练与可复现信息

6.1 模型与平台

6.2 超参与配置差异

R6 执行前的指南记录了计划参数，平台最终回报了实际运行参数。两者在 LoRA 结构上一致，但在 batch 和 max length 上有差异：

本报告在最终模型描述中采用平台回报参数，同时保留计划值作为复现实验差异。这个差异也提醒：实验报告必须记录“计划配置”和“实际运行配置”，不能只保留其中一份。

表 16: R6 模型与工作空间

项目	值
部署模型 ID	`qwen3-4b-instruct-2507-5f8261ad123d`
微调任务	`ft-202610051556-cc87`
微调输出模型	`qwen3-4b-instruct-2507-ft-202610051556-cc87`
基座	`qwen3-4b-instruct-2507`，fresh base
百炼 workspace	`ws-avxkjb2tq5lq1gwm`
训练平台	阿里云百炼 cn-beijing
推理路径	detect 直接推理；triage 追加 `TRIAGE_PURPOSE_GUIDE`

表 17: R6 计划超参与平台回报

参数	R6 指南计划	平台/评测报告回报
LoRA rank	16	16
LoRA alpha	32	32
LoRA dropout	0.005	0.005
learning rate	1e-4	1e-4
epochs	3	3
batch size	16	128
max length	2048	4096
scheduler	cosine	cosine

7 R6 评测结果

7.1 评测套件

R6 使用四类评测：

1. 冻结留出集：去泄漏的 19 条 detect + 36 条 triage；
2. 18 条真实 triage 探针：12 Confirm + 6 Reject；
3. R6 设计测试集：602 条 detect + 58 条 triage，与训练同源，只作相对参照；
4. 能力矩阵和安全库抗误报：13 条规则各一正一负，以及 10 条安全写法。

7.2 冻结留出集

detect 的 12 个安全样本中有 5 个被误报；7 个漏洞正例中有 2 个漏报。triage 的 4 条误报为 `Crypto_IO__PBES__PBES2_encrypt`、`passlib_cisco_pix`、`passlib_sun_md5_crypt` 和 `r5v-011-tneg`。4 条真实 Confirm 全部命中，但由于正例只有 4 条，Confirm recall 1.000 只能作为计数，不能作统计验收。

表 18: R6 冻结留出集结果

切片	n	recall	precision	F1	CWE acc	备注
detect	19	0.714	0.500	0.588	0.400	tp/fp/fn/tn = 5/5/2/7
triage	36	1.000*	–	–	–	FPR 0.125，accuracy 0.889，仅 4 Confirm

* Confirm recall；正例只有 4 条，故只是计数，不是统计估计。

7.3 18 条真实 triage 探针

表 19: R6 18 探针结果

指标	值
n	18（12 Confirm / 6 Reject）
Confirm recall	0.500
FPR	0.000
accuracy	0.667

六个错误全部是 Confirm → Reject，没有 Reject 被误报为 Confirm。错误分为两类：

1. **解释与裁决矛盾（3 条）**：模型解释中得出 `-> Confirm`，但 JSON 的 `verdict` 输出 Reject。例如 MD5 口令比对、弱随机 token 和 MD5 签名；
2. **幻觉出不存在的安全语境（3 条）**：模型把纯加密函数解释为“缓存键”“测试夹具生成 ETag”“缓存一致性”，而输入代码没有这些上下文。

这说明 R6 的最严重回归不是单纯漏报，而是 triage 决策链不稳定：解释能力尚在，但输出字段与安全用途推理没有完全对齐。

7.4 R6 设计测试集

表 20: R6 test split 结果

切片	n	recall	precision	F1 / FPR
detect	602（325 正 / 277 负）	0.859	0.550	F1 0.671
triage	58（28 Confirm / 30 Reject）	Confirm recall 0.571	–	FPR 0.200，accuracy 0.690

detect 的 `tp/fp/fn/tn = 279/228/46/49`，CWE accuracy 0.724。test split 与训练同源，且 detect GT 含 PyCode-Vul 模型预测噪声，因此该套结果只能作为相对参照。

7.5 13 规则能力矩阵

漏洞检出为 11/13，CWE 正确为 6/13，安全不误报为 10/13。模型能抓明显模式，但会把多个 CWE 折叠到 CWE-327、CWE-259 或 CWE-798，且对安全写法仍会误报。

7.6 安全库抗误报

10 条安全写法中有 8 条未误报，FPR 0.20。误报为：

- `cryptography.hazmat...AESGCM(key).encrypt(os.urandom(12), ...)` 被判漏洞 CWE-327；
- `nacl.secret.SecretBox(key).encrypt(msg)` 被判漏洞 CWE-327。

7.7 验收门槛

8 R6 相比 R5：恢复了什么，仍未解决什么

8.1 恢复

该对比是方向性而非严格同口径：R5 与 R6 评测切片不完全相同，R5 自身也有运行波动。R6 证明全量 fresh-base 重训可以从失败中恢复，并学到明显密码学模式的弱信号，但不证明生产级 precision 或真实 Confirm 召回。

8.2 仍存在的问题

1. **CWE 路由是明显短板**：冻结 CWE accuracy 0.400、矩阵 6/13，模型倾向折叠到 CWE-327、CWE-259、CWE-798；
2. **triage Confirm 召回退化**：18 探针 0.500，测试集 0.571，并出现解释与 verdict 矛盾；
3. **安全库仍误报**：AESGCM和 `nacl.SecretBox` 被误判；
4. **真实正例不足**：30条低于 50–100 门槛，无法支撑真实 crypto 泛化结论；
5. **冻结集统计功效不足**：detect 19条、triage 36 条且仅 4 Confirm；
6. **prompt 对齐问题**：训练数据剥离 `TRIAGE_PURPOSE_GUIDE`，推理时又追加；该引导以“非安全目的 → Reject”结尾，可能把 verdict 拉向 Reject；
7. **训练分布泛化问题**：新增 Reject、bneg、legacy、测试夹具和缓存样本过多，模型把这些语境过度泛化到真实漏洞；
8. **残余泄漏**：冻结集相对优化集干净，但合并 train/val/test 仍有继承重复。

9 结论、局限与 R7 规划

9.1 结论

R6 是一个诚实、可复现的科研基线。它验证了：

- 静态规则 + 小型领域模型 + 结构化输出可以端到端运行；
- fresh-base 全量重训可以恢复 R5 崩溃；
- 模型能学到 MD5、DES、RC4、ECB、弱 TLS 等显式模式；
- 数据来源、审核模式、SHA-256 冻结和去泄漏评测可以工程化；
- 真实数据瓶颈、样本规模和上下文标签是比“训练更久”更关键的约束。

它没有证明：

- 生产级 precision、Confirm recall 或 CWE 路由；
- 未知密码学漏洞的泛化发现能力；
- 对安全库代码的稳健误报控制；
- 模型审核等于人工 ground truth；
- 合并训练数据零泄漏。

9.2 主要局限

第一，真实 crypto 正例只有 30 条，且集中在 CWE-295 和 CWE-208。稀有 CWE 的逐类结果不稳定，无法用聚合分数掩盖。

第二，冻结集样本很小。triage 只有 4 条 Confirm，一条预测变化就能显著改变 recall；应报告绝对计数和置信区间，而不能把 1.000 当作能力证明。

第三，通用 PyCode-Vul 标签与 crypto 专用真值混在一起。2,037 条 detect 正例中只有 434 条属于目标 crypto CWE，聚合 detect 指标不能衡量 crypto 泛化。

第四，强模型审核是模型证据，不是人工证据。60 条 K3 记录和 19 条人工记录必须分开统计；K3 可以降低工作量，但不能替代独立 ground truth。

第五，提示词训练/推理不一致会直接影响 verdict。R6 的 3 条“解释正确但 verdict 相反”很可能与此有关，后续必须做 prompt 对齐或结构化字段一致性校验。

第六，残余 AST 重复说明数据继承仍有风险。未来必须按 repo/commit 分层切分，并在每次合并后重跑 exact/AST 审计。

9.3 R7 优化路线

1. **把真实正例扩到 50–100 条**：继续 GHSA/OSV/PyPA -> fix commit -> parent source -> 函数重建 -> 强模型/人工复核；按 advisory、repo、CWE 和规则统计覆盖，不只追总条数。
2. **构建真正外部测试集**：使用时间切分、仓库切分和未参与规则设计的固定 commit；评估时报告 bootstrap 置信区间，而不是单点分数。

3. **建立 train-only crypto-focus sampler**：按目标 CWE 和规则族设置最低采样配额，保护 CWE-321/329/916/256/208 等稀有类，不触碰 val/test。

4. **对齐训练与推理 prompt**：要么训练时也加入 purpose guide，要么在推理时去掉 guide；同时增加 JSON schema 校验和“解释与 verdict 一致”检查。

5. **增强安全负例质量**：继续从固定 commit 安全库提取，但必须先检查调用上下文；重点补充 AESGCM、SecretBox、legacy 格式、测试/缓存/ETag 语境，避免把弱原语用途过度泛化。

6. **改进 CWE 目标**：为每个目标 CWE 建立 per-CWE precision/recall，加入 CWE 层级或规则映射辅助，不让模型自由坍缩到 CWE-327。

7. **分离合成、规则夹具和真实记录的评价层**：规则夹具用于工具回归，真实记录用于质量评估，合成样本用于覆盖，三者不能共用一个 headline。

8. **升级基座或继续预训练**：在数据质量达到门禁后，再考虑更强基座、更长上下文或领域继续预训练；基础模型升级不能弥补错误标签。

9. **建立人工复核质量机制**：双人复核一个小抽样，统计一致性；对 K3 与人工分歧进行误差分析，逐步把高置信模型标签转为抽检标签。

10. **继续做 leakage 与许可证治理**: Semgrep 规则夹具遵守 Semgrep Rules License v1.0, Bandit 遵守 Apache-2.0；对外发布时保留来源和许可元数据。

10 复习问答与面试要点

10.1 常见问题

为什么选择密码学代码安全?

密码学代码有明确 API、明确 CWE，也有非常强的上下文依赖。它既能评估静态规则，也能暴露 LLM 在安全语义、幻觉和数据质量上的问题。

为什么把 Semgrep 和 LLM 组合?

Semgrep 提供高召回、可定位和可解释的候选；LLM 提供用途判断和误报过滤。单独使用 Semgrep 会产生 legacy 兼容、缓存、测试夹具等有效误报；单独使用 LLM 缺少稳定定位和可复现规则。

为什么不直接用大模型 API，而要微调小模型?

小模型更便宜、可部署、可研究领域适配，也方便与静态规则结合。项目把大模型放在数据审核环节，而不是生产推理路径。

本项目最重要的技术发现是什么?

第一，fresh-base 全量重训比增量续训稳定；第二，数据质量门禁比样本数量更重要；第三，真实 crypto 正例和人工/模型来源标记是核心瓶颈；第四，prompt 对齐会直接影响 triage verdict；第五，小正例数让 recall 估计极不稳定。

R6 是失败还是成功?

R6 成功恢复了 R5 的 detect 崩溃，并形成了可复现的数据与评测流水线；但它没有通过 precision、FPR 和探针准确率门槛，所以只能定义为临时科研基线，而不是成功发布的生产模型。

10.2 实验中的注意事项

- 绝不在报告里混用轮次指标，每个数字必须标注轮次、切片和样本量；
- 报告 Confirm recall 时必须同时报告正例数；
- 不把 `verified=false` 的规则夹具写成真实漏洞；
- 不把 K3 标签写成人工标签；
- 不声称零泄漏；
- 不把 R6 test split 的边界 precision 当作生产可用证据；
- 训练前核对 manifest 哈希，训练后不再修改 train/val/test；
- 超参要同时保存计划值和平台实际值；
- 每次扩充真实数据都按法规、许可证和来源链检查。

A 复现命令与关键文件

A.1 构建与冻结

```
python3 scripts/build_round5_final_dataset.py --emit
python3 scripts/harvest_real_crypto_candidates.py \
  --advisories data/external/advisory-database/advisories
python3 scripts/screen_real_crypto_candidates.py --emit
python3 scripts/rebuild_candidate_functions.py --dry-run
GITHUB_TOKEN=ghp_xxx python3 scripts/rebuild_candidate_functions.py
python3 scripts/audit_real_crypto_with_llm.py --estimate-only
python3 scripts/annotate_real_crypto.py \
  --candidates data/round6/candidates/real_crypto_candidates_uncertain.jsonl \
  --reviewer reviewer-kimi --open
python3 scripts/build_round6_final_dataset.py --emit
python3 scripts/audit_dataset_leakage.py --write-frozen
python3 scripts/build_dataset_metadata.py
```

A.2 评测

```
python3 scripts/run_round6_eval.py
python3 scripts/run_round6_test_split.py
python3 scripts/run_round6_capability.py
```

A.3 关键产物

- `data/round6/eval/RESULTS.md`：R6 完整评测结论；
- `data/round6/eval/benchmark_r6.json`：冻结 detect + triage 逐条预测；
- `data/round6/eval/triage_probe_results_r6.json`：18 探针与解释；
- `data/round6/eval/test_split_r6.json`：660 条测试集逐条预测；
- `data/round6/eval/capability_r6.json`：13 规则矩阵和安全库探针；
- `reports/data_quality/model_audit_k3_cost.json`：K3 实际成本；
- `reports/data_quality/model_audit_k3_import.json`：K3 导入与跳过原因；
- `data/round6/final/r6_manifest.json`：源文件和制品哈希；
- `data/round6/validation_frozen/manifest.json`：冻结评测集与去泄漏说明；
- `reviewer/README.md`：浏览器二元审核说明。

B 术语表

表 21: R6 13 规则能力矩阵

规则	期望 CWE	漏洞检出	预测 CWE	安全不误报
CRYPTO-001 MD5	CWE-327	✓	CWE-327	✓
CRYPTO-002 SHA-1	CWE-327	×	–	✓
CRYPTO-003 DES	CWE-327	✓	CWE-327	✓
CRYPTO-004 RC4	CWE-327	✓	CWE-327	✓
CRYPTO-005 AES-ECB	CWE-327	✓	CWE-327	✓
CRYPTO-006 可预测 IV	CWE-329	✓	CWE-327	✓
CRYPTO-007 硬编码密钥	CWE-321	✓	CWE-327	×
CRYPTO-008 弱随机	CWE-338	×	–	✓
CRYPTO-009 弱密钥长度	CWE-326	✓	CWE-326	✓
CRYPTO-010 不安全 TLS	CWE-326	✓	CWE-326	✓
CRYPTO-011 明文口令	CWE-256	✓	CWE-259	×
CRYPTO-012 弱 KDF 参数	CWE-916	✓	CWE-798	×
CRYPTO-013 时序比较	CWE-208	✓	CWE-259	✓

注：能力夹具把 CRYPTO-010 的期望 CWE 标为 CWE-326，而规则表中该规则映射为 CWE-295；这一夹具与规范不一致本身也是 R7 需要修复的数据一致性问题。

表 22: R6 验收门槛

门槛	目标	R6 状态
detect precision	> 0.54	× 冻结 0.500；测试集 0.550 临界
domain FPR	≤ 0.06	× 冻结 0.125、测试集 0.200、安全库 0.20
triage Confirm recall	仅报告	冻结只有 4 条 Confirm，无统计意义
18 探针 accuracy	≥ 0.95	× 0.667
真实仓库 FPR	不差于 R3/R4	× 未解决

表 23: R5 与 R6 的方向性对比

指标	R5	R6	解释
detect F1	0.000	0.588 冻结 / 0.671 测试	从 detect 崩溃中恢复
detect recall	0.000	0.714 冻结 / 0.859 测试	基础检出能力回来
triage FPR	0.368	0.125 冻结	冻结集误报下降，但仍高于目标
triage accuracy	0.658	0.889 冻结	冻结集上改善，但样本构成不同
Confirm recall	3 正例上 1.000	18 探针上 0.500	说明小正例 recall 不能外推

术语	含义
SAST	静态应用安全测试，不运行程序分析源码
detect	仅输入代码，判断漏洞和 CWE
triage	输入代码和 finding，判断 Confirm/Reject
CWE	通用缺陷类型枚举
CSPRNG	密码学安全伪随机数生成器
AEAD	同时提供机密性与完整性的认证加密
KDF	从口令或主密钥派生子密钥的函数
LoRA / QLoRA	低秩适配 / 量化低秩适配
fresh-base retrain	从原始基座用全量合并数据重训
catastrophic forgetting	新训练导致旧能力退化
FPR	真负例中被错误判为阳性的比例
verified=false	尚未经过独立真实性审核
`review_mode`	审核来源：`manual` 或 `model`